Report on Entity Component System

Motto: A brief fun introduction to gaming world with some tea on architectural systems of an engine and various ways the concept has been utilized

Author/Creator of the Report: G Navaneeth Singh (ML and tech enthusiast)

Referenced resources links have been mentioned, for Better understanding purposes and copyright claims

For more info contact :
navaneethsingh73@gmail.com

Entity Component System:

What is a entity component system?

- Entity component system is a way for us to separate the data component , entity component and identity component.

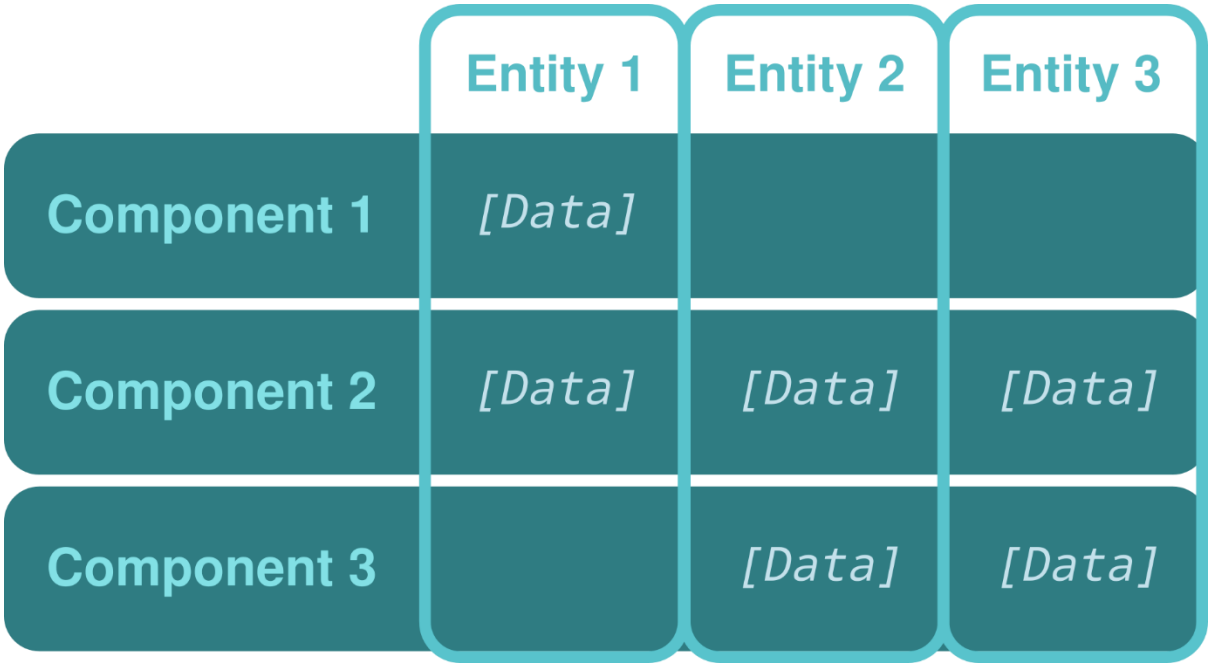
- What consists of the data component , suppose for example lets consider a game environment , a game has several assets be the different characters , different accessories or even different storylines these all make up the data component

- Entity component: Here the data is grouped in separate entities based on their usage , like if the data used in positioning it will stored in “Positioning Entity” , like the Entity component is basically used a way segregate the data based on their usage.

- Identity Component: This component just contains unique “id’s” associated which each component. Like suppose if I want to look for different data component associated with id “406” , we can easily track different component associated with this id, also note that the “id’s” are completely unique.

- There are several advantages of creating a entity component system over traditional method of creating objects , also lets say some several advantages using this methodology to solve problems many software applications are based on this as well.

Entity Component System:



Memory Layout strategies of a Ecs system :

- There are primarily 2 startegies of storing the data components 1).Archetype Memory Layout , 2.) Sparse set memory layout

Archetype Memory Layout

Under archetype memory layout there different type of arrangement methods

1.)Chunk system:

- The Chunk system way storing is that grouping all the data of a particular object and storing it contiguously

- This way all the data of particular object is available at a particular memory address

2.) Structure of Array (SOA):

Entity Component System:

- Insteading of storing the data side by side for different obejcts , we try to store the data of components as structure of arrays.

- According to the “entity id” of diiffernt objects we can access the values of different component for that particular object

Memory Flow:

- The Memory Flow in this type of approach of storing how it does work is , let’s say we want to access “Position Component” of entity number 1 the entire the Position component is loaded on to the L1 cache.

- CPU pre fetcher see’s the linear pattern and loads several blocks of data for particular component before the system even asks for it.

Applicable or Currently Under Use:

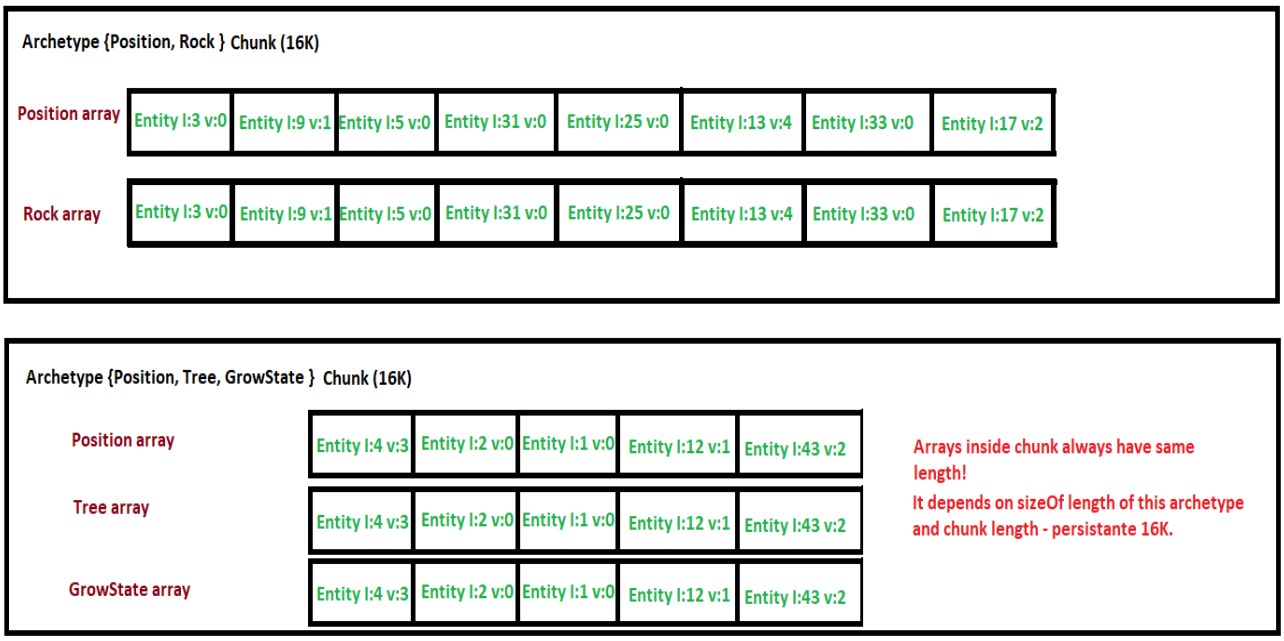
- Archetype of Memory Layout is currently being used by several gaming engines popular one’s include unity , unreal engine, bevy .

- Also the another major things people often confuse saying that “most of gaming engines are using C++”. Well this might be true or else could also be a hot take.

Entity Component System:

- Engines are Bevy are using “Rust “ to build the open source engines.

- Also the “Different Companies Use this method of arranging the data” for “different benefits”.



2.) Sparse Set Memory Layout:

- In this method of arrangement or storage strategy the component are stored and modified according to their “entity id”.
  Ex: pos_x[entity_id]=value;

Entity Component System:

- This method might be feasible or scalable for a system which has lesser number of entities , but in a gaming system where there are ton of entity need to created , accessed , destroyed this method might pose a threat to “speed of the system”.

  Ex: After deleting some entity id’s , Active entities: 0, 1, 5, 8, 14, 23, 97, 204, 1047, 8832… might look like this.

- If you index directly by entity ID, your component array needs 8,833 slots just to hold 10 active entities. 8,823 slots are empty holes — wasted memory. Worse, iterating the array means checking every slot including the empty ones — the CPU still loads those empty cache lines. The contiguous access advantage is gone.

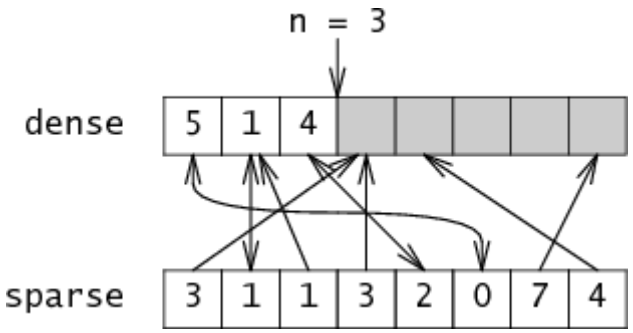
Memory Flow in Sparse Set :

- The memory flow in is divided into 2 core structures called as *)Dense array and *)Sparse array.

- Sparse array – This array contains the data which is indexed along with the entity id. Actual data is contained here.

- Dense Array- This contains the Entity id and also manages the updation or deletion of the “data components” of the entity id.

Entity Component System:



Pictorial representation sparse set

Applicable or currently being used:

- This type of “memory arrangement” is currently being used by “Minecraft” in bedrocks and arcane studios in “deathloops”.

- EnTT a open source C++ Ecs library uses this methodology. Bevy engine uses a hybrid system of both “archetype” and “sparse sets”.

EnTT
GAMING MEETS MODERN C++

BEVY

Entity Component System:

Disadvantages: 1.) The sparse[] array must be sized to the maximum possible entity ID. If your game ever creates entity #100,000, the sparse array needs 100,000 slots — even if only 500 entities are currently alive.

Active entities: 0, 1, 5, 8, 97, 204, 99999

sparse[] size: 100,000 slots

Actually used: 7 slots

Wasted: 99,993 slots sitting empty

In a game with millions of entity IDs over its lifetime (created and destroyed continuously), the sparse array grows large and never shrinks. For embedded systems or memory-constrained platforms like Nintendo Switch, this is a real problem.

2.) Disadvantage 2 — Two Cache Misses on Every Lookup

Every single component lookup requires hitting two separate memory locations:

Step 1: Read sparse[entity_id] — cache miss #1 (sparse array)

Step 2: Read data[dense_index] — cache miss #2 (data array)

The sparse array and the dense/data arrays are stored at completely different memory addresses. The CPU loads a cache line from sparse[], extracts one index value, then has to jump to a completely different memory region to read data[]. That second jump is almost always a cache miss because nothing about the sparse array access tells the prefetcher where the data array access will land.

Entity Component System:

Compare this to a direct array lookup where you pay exactly one cache access. Sparse Sets inherently double the cache pressure for random lookups.

A Good article to refer to regarding a more detailed representation:

https://ajmmertens.medium.com/building-an-ecs-storage-in-pictures-642b8bfd6e04

Traditional Method of Creating Objects v/s Entity Component system:

- The traditional method was “Object Oriented way” of creating objects.

- The main principle behind this method create a general class based on the create unique “inherited classes” which has inherited characterstics from the “general class”.

- In this type of creation the data is stored in the form of “Array of structures”(AOS Method), when you create a object this system allocates a fat block of memory to it.

- Data Bundling – all the component data are stored side by side , suppose we have several variables (like position,velocity,current state etc) all these values are stored side by side.

Entity Component System:

Disadvantages of OOP method compared to ECS :

1.) Maximum Efficient Cache – In the Oop method of creation the data is scattered over the heap memory section resulting in higher cache misses

- Whereas as in ECS system the SOA method ensures the data is tightly coupled for better cache performance lessening the number of misses.

2.) Solving the "Inheritance Hell" (Composition)

In traditional systems, you define what an object *is* (e.g., "A Dragon is an Actor"). This becomes rigid and breaks down if you want a Dragon that behaves like a Treasure Chest.

- The ECS Edge**:** You define what an entity *has* (Composition).
- Flexibility**:** If you want a "Healing Potion" to suddenly fly and shoot fire, you simply add the FlightComponent and CombatComponent to its ID. You don't have to rewrite a single line of class code or change a hierarchy.

3.) Native Multithreading & Parallelism

Parallel programming (using multiple CPU cores) is notoriously difficult in traditional OOP because objects often depend on each other, leading to "Race Conditions" or crashes.

- The ECS Edge: Since **Systems** are decoupled and operate on specific arrays of data, the ECS manager can easily see which systems are "thread-safe."
- Automatic Scaling**:** A system that processes Physics can run on Core 1 while a system processing Animation runs on Core

2 at the same time, because they are touching different memory arrays.

Code Comparison:

OOP Method :-

```
#include <iostream>
#include <vector>
#include <chrono>

struct Entity {
    float x, y;
    float vx, vy;
    int id;
    char padding[64]; // Simulating a "fat" object with extra data

    void update(float dt) {
        x += vx * dt;
        y += vy * dt;
    }
};

void runOOP(int count) {
    std::vector<Entity> entities(count);
    // Initialize
```

Entity Component System:

```
    for(int i = 0; i < count; ++i) {
        entities[i] = {0, 0, 1.0f, 1.0f, i};
    }

    auto start = std::chrono::high_resolution_clock::now();

    // The Logic: The CPU must jump through "fat" objects
    for(int i = 0; i < count; ++i) {
        entities[i].update(0.016f);
    }

    auto end = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double, std::milli> diff = end - start;
    std::cout << "OOP Time: " << diff.count() << " ms" << std::endl;
}
```

Ecs Method :

```
#include <iostream>
#include <vector>
#include <chrono>

struct PositionComponent {
```

Entity Component System:

```
    std::vector<float> x, y;
};

struct VelocityComponent {
    std::vector<float> vx, vy;
};

void runECS(int count) {
    PositionComponent pos;
    VelocityComponent vel;

    // Initialize contiguous arrays
    pos.x.resize(count, 0);
    pos.y.resize(count, 0);
    vel.vx.resize(count, 1.0f);
    vel.vy.resize(count, 1.0f);

    auto start = std::chrono::high_resolution_clock::now();

    // The Logic: Pure linear memory access
    float dt = 0.016f;
    float* px = pos.x.data();
    float* py = pos.y.data();
```

Entity Component System:

```
    float* pvx = vel.vx.data();
    float* pvy = vel.vy.data();

    for(int i = 0; i < count; ++i) {
        px[i] += pvx[i] * dt;
        py[i] += pvy[i] * dt;
    }

    auto end = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double, std::milli> diff = end - start;
    std::cout << "ECS (SoA) Time: " << diff.count() << " ms" <<
std::endl;
}
```

TIME Comparison taking a general case of creating two objects:

```
navaneeth@LAPTOP-977VC2OP:~$ gedit OOP.cpp
navaneeth@LAPTOP-977VC2OP:~$ g++ OOP.cpp -o OOP
navaneeth@LAPTOP-977VC2OP:~$ ./OOP
OOP Time for 100000 entities: 0.741799 ms
navaneeth@LAPTOP-977VC2OP:~$
```

V/s

```
navaneeth@LAPTOP-977VC2OP:~$ gedit ECS.cpp
navaneeth@LAPTOP-977VC2OP:~$ g++ ECS.cpp -o ECS
navaneeth@LAPTOP-977VC2OP:~$ ./ECS
ECS (SoA) Time for 100000 entities: 0.461056 ms
navaneeth@LAPTOP-977VC2OP:~$
```

Entity Component System:

- And remember this is the difference only for the creation of 2 objects , if we were to create millions of objects the difference will be of night and day.

But still why did we use the “OOP way for such longer period of time :-

1. The "Boilerplate" and Complexity Burden

- In the traditional method, creating a new behavior is simple: you add a variable and a function to a class. In ECS, the same task requires:
- Defining a new Component (Data).
- Creating a new System (Logic).
- Registering the system in the Dispatcher (Execution order).
- The Result**:** Small features take more code to set up. This makes ECS overkill for small projects or simple games where performance isn't an issue.

2. Difficult Inter-Entity Communication

- In OOP, an object can easily call a method on another object: target.TakeDamage(10). In ECS, components are just raw data; they don't "do" anything.
- The Problem**:** If Entity A needs to affect Entity B, you usually can't just call a function.
- The Solution: You have to use Event Buffers or Tags. You add a "DamageRequest" component to Entity B, and then a separate system must find that tag, process the damage,

and remove the tag. This "indirect" way of thinking can be very frustrating for developers used to direct interaction.

3. The Learning Curve & "Brain Shift**"**

- Traditional OOP follows how humans naturally see the world (a car is a car). ECS follows how the CPU sees the world (a car is a collection of floats and ints).
- The Challenge: Most developers are trained in OOP from day one. Shifting to Data-Oriented Design requires unlearning years of habits regarding inheritance and encapsulation.
- Debugging**:** It is harder to debug. In OOP, you can pause the game and inspect the "Player" object. In ECS, the player's data is scattered across five different memory arrays, making it harder to get a "complete picture" of a single entity's state at a glance.

4. Archetype Fragmentation (Memory Waste)

- While Archetypes are designed for speed, they can lead to memory waste if your entities are too unique.
- The Problem: If you have 100 entities and each has a slightly different combination of components, the engine creates 100 different Archetypes.
- The Result: Instead of large, efficient chunks of memory, you end up with many tiny, nearly empty chunks. This negates the cache benefits of ECS and can actually make it slower than traditional methods in specific "chaotic" scenarios.

Entity Component System:

Conclusion of the competition of OOP v/s ECS:

- Even with all these advantages of OOP , still ECS is best approach why?. The answer majorly lies in “Cache locality problem”, speed when working on games which has denser environments with large number of entities the OOP method clearly sucks.

Subsystems of an ECS :

1.) Component Manager(*Librarian of ECS) :

- The component manager is also called as the librarian of the Ecs , just like how a librarian manages books , the component manager manages the memory management of the data.

- Storage Allocation – It manages the storage of the chunks of memory , the structure of arrays (*SOA).

- Data Locality – Whenever a new entity is created it makes the data to be contiguous in order to optimize cache.

- Mapping – Effective management of entity data , suppose if the system wants entity 6’s data easy retrieval is easily done

2**.)** The Entity Manager (The Registrar)

The Entity Manager is a lightweight subsystem that handles the "Identity" of every object in your simulation.

- ID Generation: It issues unique integer IDs.

Entity Component System:

- Recycling: When an entity is "destroyed," the manager doesn't just delete the number; it marks it as "stale" and eventually reissues it to a new object to save memory (often using a Generation ID to prevent old pointers from working).
- Tagging**:** It tracks which "Tags" or "Labels" are currently active on which IDs, allowing for lightning-fast filtering.

3.) The System Dispatcher:

This is arguably the most complex subsystem. It controls **when** and **how** the logic runs.

- Execution Order**:** In a complex simulation, System A (Physics) must run before System B (Collision). The Dispatcher manages this hierarchy using a **Task Graph**.
- Multithreading : The Dispatcher looks at the "signatures" of the systems. If two systems don't touch the same data, the Dispatcher automatically sends them to different CPU cores to run in parallel.
- Delta Time Management: It ensures that every system receives the correct "tick" or time-step information to keep simulation speeds consistent.

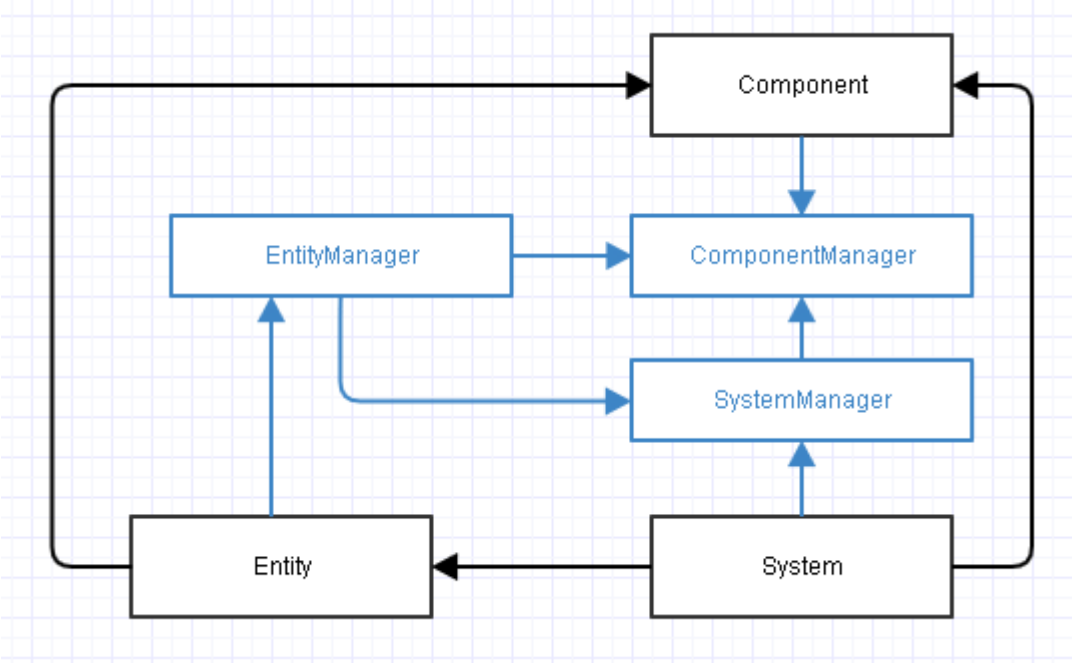
4.) The Query/Filter Engine (The Search Engine)

Systems don't manually search for entities; they ask the Query Engine for a specific "view" of the world.

- Signatures: A system defines a signature like include(Position, Velocity), exclude(StaticTag).

Entity Component System:

- Caching Results: The Query Engine often caches these results. Instead of searching through 100,000 entities every frame, it keeps a "live list" of entities that match the criteria.
- Reactive Queries: It can trigger specific logic only when a component is *added* or *removed* (e.g., "Play a sound only when the DeathComponent is added").



General Purpose Utilization of ECS idea :-

- The ECS ideas has been utilized by many fields as blueprint for reasons like cache locality , tightly coupled memory , SOA method of storing of data.

- The core concepts like this have been used for various different reasons and methods to fullfill their needs.

  1) Columnular Database

Entity Component System:

2)Numpy and Pytorch

3)LAPACK/BLAS(*Linear Algebra Library)

Columnular Databases :-

- These databases follow the exact same approach of storing different column as “contiguous array” similar fashion to the ECS system.

- How do these databases differ or are advantageous compared to regular db’s ?

- When u want to retrieve a specific column from db’s suppose lets take this example of getting salary of the person whose id=320
  Query:
  SELECT salary
  FROM Employee
  WHERE id=300;

- How this data is loaded is that , since we need only one field for each cache cycle it loads 8 bytes per cycle wasting the other 56 bytes(*Since in Each cycle 64 bytes are loaded).

- This wastes time as well as memory , as number of cache misses increase in a regular db retrieval.

Entity Component System:

- But in columnular db since each columns are stored as contiguous array each time the entire 64 bytes are loaded on , decreases cache misses improves retrieval speed.

- Another example :- Suppose Row 0: [id:1, name:'Alice', age:25, salary:50000, dept:'Eng']
- Row 1: [id:2, name:'Bob', age:30, salary:60000, dept:'HR']
- Row 2: [id:3, name:'Carol', age:28, salary:55000, dept:'Eng'](1 million rows)
- Each row is one contiguous bundle in memory — exactly like an AoS game object.

- Now imagine this query:
- Sql

- SELECT AVG(salary) FROM employees;

- The database needs only the salary column. But in row storage, it has to load every single row — name, age, dept, everything — just to extract the salary from each one. It reads maybe 200 bytes per row but uses only 8. The rest is wasted. For 1 million rows, that's loading 200MB from disk just to use 8MB. Pure cache waste, identical to the OOP skeleton problem.

- Even though this database has several advantages it has some issues too, like if u want to enter a new tuple to the database in “regular db” this is easily done , but in a columnular db we have to load each column’s contiguous array each time a new entry has to happen.

Entity Component System:

- The columnular database as a whole is not a new topic at all , many articles as early as 2005 have mentioned regarding the research on this topic.

  Some really good research papers on this topic which do detail the topic in a good manner

  https://www.irjet.net/archives/V7/i4/IRJET-V7I41213.pdf

  https://web.stanford.edu/class/cs345d-01/rl/cstore.pdf

2.)Numpy and Pytorch :

- Numpy and Pytorch follow the same philosophy as well instead of storing the training data as objects , these libraries store as Structure of 2D Array.

- Storing the training data as Structure of Array really helps to optimize cache achieving higher speeds of computation.

3.)BLAS/LAPACK :

- All the scientific computing matrices are stored as Structure of combined arrays.

- Again improving cache speeds compared to storing of matrices as a object.

Entity Component System:

Articles referenced while making of this report :

*)https://austinmorlan.com/posts/entity_component_system/

*)https://discussions.unity.com/t/archetype-memory-layout-alternatives/864724/5